INF2004 Embedded Systems Programming

Week 8 Tribunal 1 Template

Architecture Defence & Integration Readiness | AI ideation, live update, AI-vetted gap check

Item	Details
Purpose	Defend Week 6 architecture, justify changes, and show readiness for partial integration.
Core workflow	Pre-submission AI ideation -> live Zoom-updated version -> final AI-vetted gap check
Assessment focus	Evidence, engineering judgement, negotiation, and ability to identify missing work
Submission format	Completed document plus Zoom recording and evidence links

Important: this submission has THREE versions inside one document

- Version A: Pre-tribunal AI ideation. Each buddy uses AI to ideate their subsystem position before Zoom.
- Version B: Live Zoom-updated version. Design decisions are updated during the Zoom discussion through challenge, negotiation, and constraint response.
- Version C: Final AI-vetted gap check. After discussion, AI is used as a reviewer to identify missing assumptions, risks, tests, or unclear decisions.
- Marks are not awarded for blindly following AI. Marks are awarded for critical evaluation and human engineering judgement; meaning, I want to see you guys discuss during zoom.

Team Information

Item	Details
Group ID	P2-8
Repository Link	https://github.com/timothyckl/INF2004-P2-08-RobotCar

Buddy	Student ID	Primary subsystem / persona
Buddy 1 (Hong Yih)	2501110	WiFi and MQTT communication, including topic and message design, telemetry publishing, command reception and forwarding, heartbeat reporting, and connection recovery.
Buddy 2 (Dalton)	2504003	Motor and encoder integration, PID speed control, distance control and repeatable turns.
Buddy 3 (Edmund)	2501578	IR sensor calibration, line following, barcode decoding and navigation command generation.
Buddy 4 (Khang)	2503012	IMU calibration, motion and hump detection, highest-peak measurement and event reporting.
Buddy 5 (Timothy)	2501530	Servo-mounted ultrasonic scanning, obstacle profiling, avoidance planning and line reacquisition.

Version A: Pre-Tribunal AI Ideation

Instructions

- Each buddy must use AI before Zoom session to prepare subsystem-specific arguments, risks, and negotiation points.
- The AI output must be critiqued. Identify what is useful, unrealistic, incomplete, or requires validation.
- For Tribunal 1, compare against the Week 6 architecture freeze. For Tribunal 2, compare against actual integration evidence.

Buddy	AI prompt used	Useful AI ideas	Weak / unrealistic AI ideas	Position before Zoom
Buddy 1	Review the Week 6 WiFi/MQTT plan against the professor's feedback. Suggest arguments, risks and negotiation points for commands, telemetry and reconnection; critique them. Draft for buddy review; planning only.	Keep networking separate from vehicle control. Define MQTT topics, payloads, heartbeat and command validation. Copy shared state briefly, then publish outside the mutex. Test control timing and telemetry delay during reconnects; reconnecting must not restart movement.	Lowest priority alone does not prove that WiFi work cannot delay control. Topics, payloads, command validity by mission state and measurable telemetry latency are undefined. Shared-state locking and retry behaviour need testing. No implementation or test evidence yet.	Agree START/STOP/RESET rules and message limits with Dalton (Buddy 2: Motor and Encoder/Controller). Define telemetry fields with all sensor owners. Agree a latency target and test reconnection alongside control and scanning. Keep A1/A2 open; join the shared power test.
Buddy 2	Review the Week 6 controller, motor and encoder plan against the professor's feedback. Suggest arguments, risks and negotiation points on timing, interrupts, STOP and power; critique them. Draft for buddy review; planning only.	Keep one Vehicle Controller, one motor-PWM writer (Motion Control) and a latched STOP. Set measurable distance and turn error targets. Use a single GPIO interrupt dispatcher for encoder, ultrasonic ECHO and proposed IR edge interrupts, and evaluate the combined interrupt load. Keep continuous readings separate from discrete events, with defined message fields, queue limits and overflow handling. Test control and STOP	AI treats task priorities and short ISRs as if they guarantee timing; they don't, so maximum interrupt load, task execution times and queue behaviour still need measuring. Pin conflicts (UART0 console on GP0/GP1, demo LED on GP16, ADC on GP26 to GP28) are documented but not yet confirmed on the board. Motor stall and servo sweep current may trip the power bank or reset the board. A3, A4 and A11 remain untested.	Proposed targets for team review: distance within ±2 cm over 50 cm; 90° turns within ±5°; U-turns within ±10°; motors commanded off within one 20 ms control cycle of STOP being set. Estimate the combined encoder interrupt rate using maximum wheel speed, encoder resolution, gear ratio where applicable, and the selected GPIO edges. Confirm against measured ISR execution time. Agree junction-command timing with Edmund (Buddy 3) and stop confirmation, bypass and cancellation messages with Timothy (Buddy 5).

		timing while WiFi is reconnecting.		Clarify whether remote STOP requires a separate urgent signalling mechanism to prevent delays when remote_q contains pending commands. Confirm that network reconnection cannot automatically restart movement or execute stale commands. Agree motion_q command and result formats, including IDs, units, limits and completion/fault reporting, during Zoom. Plan encoder-load, STOP, WiFi-reconnect and motor-stall/servo-sweep power tests.
Buddy 3	Review the Week 6 IR, barcode and navigation plan against the professor's feedback. Suggest arguments, risks and negotiation points on sampling, junctions and line return; critique them. Draft for buddy review; planning only.	Use timestamped GPIO edges or encoder snapshots for barcode capture; test at target speed. Define junction detection and hold a decoded command until its junction. Keep perception separate from motor control. Define how original-line reacquisition is confirmed after a bypass.	Do not assume 20 ms polling will capture narrow barcode bars at operating speed. Junction detection, delayed command execution and original-line confirmation are not defined. GPIO interrupt sharing and event delivery need agreement. A5/A6 and barcode read success at a stated speed remain untested.	Propose testing edge-based barcode capture early and setting a read-success target at a stated speed. Agree junction recognition and command timing with Dalton (Buddy 2: Motor and Encoder/Controller), and original-line confirmation with Timothy (Buddy 5: Obstacle Scan). Propose lighting, speed and bypass-return tests; A5/A6 stay open.
Buddy 4	Review the Week 6 IMU/terrain plan using the professor's feedback. Suggest arguments, risks and negotiation points for sampling, hump-height estimation and event delivery; critique them. Planning only; no implementation or tests yet.	Propose a defined IMU sample period and an explicit method for estimating hump height in mm. Compare estimates with measured humps, separate vibration from real humps and retain the run maximum. Continuous readings and discrete terrain events need clear delivery rules.	A reliable height-estimation method is not yet established for the accelerometer-only GY-511. Noise, tilt and integration drift may affect estimates. Low priority and 'rate to validate' do not prove adequate sampling. Height error, execution time and false detections remain unmeasured; A7/A8 stay open.	Define the sampling period, height calculation and acceptable height error before claiming accuracy. Agree event formats and timing with Dalton (Buddy 2: Motor and Encoder/Controller), and telemetry fields with Hong Yih (Buddy 1: WiFi/MQTT). Propose stationary, flat-motion and known-hump tests under load, followed by a review of whether the method is adequate.

Buddy 5	Review Buddy 5's Week 6 obstacle scanning and avoidance plan. Propose subsystem arguments, risks and negotiation points; critique the suggestions. Planning only: no implementation or tests yet.	Keep ultrasonic and servo ownership in Obstacle Scan. Scan after confirmed stop; report a bypass plan to the Vehicle Controller. Use bounded echo waits and reject invalid profiles. Agree stop confirmation and plan cancellation with Buddy 2, and line-reacquisition feedback with Buddy 3.	The plan does not yet define broad and fine scan steps, scan angles, servo settling time, obstacle-width calculation or a measurable detection distance. Message contents, age limits and queue-full handling remain undefined. We also need criteria for confirming the original line, scan timing under load and power stability. Stopping adds delay; a missing echo does not prove clearance. No implementation or test evidence yet.	Retain stop-before-scan and controller-only motion authority. Agree broad/fine scan steps, angles, settling time and width calculation. With Dalton (Buddy 2: Motor and Encoder), define stop confirmation, bypass messages/cancellation, detection distance and clearance margins; with Edmund (Buddy 3: Line and Barcode), define how to confirm the original line. Propose tests for scan accuracy, safe bypass and line return, control timing during Wi-Fi reconnection, and power stability with motors and servo active. Keep A9, A10 and A12 open; coordinate the shared A11 power test. All proposals need team review and testing.

Week 6 Architecture Defence Snapshot Before Zoom

Select only **5 to 7** important Week 6 architecture claims that affect integration readiness. Do not repeat the entire Week 6 document. Each row should defend one claim using evidence, remaining weakness, and the buddy responsible. One smart-toaster example is shown; replace it with your own project claim.

Week 6 artefact	What the team claimed in Week 6	Evidence available now	Still weak or untested	Buddy defending it
Task table and interaction diagram	One Vehicle Controller owns mission state; only Motion Control writes motor PWM.	Week 6 task table, interaction diagram and statecharts document ownership. Professor feedback explicitly supports this architecture. Design evidence only; no implementation or test results.	Ownership still has to be enforced in code. Distance and turn targets are proposed (±2 cm over 50 cm, ±5° per 90° turn) but not yet agreed or measured.	Buddy 2
Interaction diagram and	A latched STOP flag bypasses ordinary command queues;	Week 6 safety flags and restart transitions describe the mechanism.	STOP response time and stale-command rejection have not been	Buddy 2

system statechart	recovery requires authorisation and stale-command removal.	Professor feedback praises the STOP and restart rules; behaviour is not yet demonstrated.	measured. Need to test with motion_q full and during WiFi reconnection. Proposed target: motors commanded off within one 20 ms control cycle.	
Task interaction diagram	Sensor tasks report through queues and flags; the controller decides vehicle actions.	Week 6 documents sensor_q, scan_q, controller_flags and ownership. This supports separation of responsibilities, not reliable delivery under load.	Professor feedback flags continuous samples mixed with discrete events in sensor_q. Plan: keep latest readings in shared state and use the queue only for events. Message fields, queue size, data age limit and overflow handling still need agreeing and load testing.	Buddy 2; Buddies 1 to 5 agree interfaces
Wireless task and statechart	WiFi/MQTT loss does not prevent essential autonomous sensing and control.	Week 6 gives Wireless Comms lowest priority and releases state_mutex before publishing. Independence is a documented design goal, not a measured result.	Define topics, payloads, state-based command validation and telemetry latency. Test control timing during reconnects and revise if needed. Planning was prioritised; no implementation/test evidence yet.	Buddy 1
Line Sensor task and statechart	Line/barcode sensing reports results to the controller and does not write motor commands directly.	Week 6 interaction diagram and Subsystem 3 statechart establish the boundary. Professor feedback identifies sensing gaps while the ownership rule remains documented.	20 ms polling is too coarse for barcode capture at target speed. Define edge capture, junction recognition and holding commands until the junction. Set read-success targets; validate and iterate through actual tests.	Buddy 3
IMU task and Subsystem 4 statechart	A dedicated IMU/Terrain task monitors terrain and retains the highest hump peak for the run.	Week 6 task table and statechart describe monitoring, peak retention and fault handling. These are planned behaviours; no measurement evidence exists yet.	Specify sample period, accelerometer-only height calculation and acceptable error in mm. Test known humps, vibration and task timing; revise the method based on results. A7/A8 remain open; actual testing has not started.	Buddy 4
Obstacle task and Subsystem 5 statechart	Scanning starts after confirmed stop; Obstacle Scan proposes a bypass while the controller	Week 6 task interfaces and obstacle statechart define stop-before-scan, sensor/servo ownership and controller authority.	Define coarse/fine stages, angles, settling time, width estimation, detection distance, clearance and original-line confirmation.	Buddy 5

	commands motion and coordinates line return.	Safe bypass performance is not yet demonstrated.	Validate A9/A10/A12 through scan and bypass tests, then iterate. Planning was prioritised before building; no actual test evidence yet.	

Shared Resource Allocation Matrix

Enter at least **3 shared resources**, or more if the project has additional shared buses, timers, interrupts, actuators, sensors, queues, event groups, or shared data. A resource is "shared" if more than one task, module, interrupt, or buddy can affect it.

Shared resource	Used by task(s)	Owner task / module	Access type	Contention risk	Mitigation	Evidence needed
sensor_q	Line, Obstacle, IMU, Motion; Controller	Vehicle Controller receives the messages. Each sending task is responsible for its own message contents.	Sensor tasks add results to the queue; Vehicle Controller reads and handles them.	Several tasks send readings into the same queue. Frequent readings may fill it or make an important event wait behind old data.	Keep the newest continuous reading in shared state and use the queue for separate events. Agree message fields, queue size, acceptable data age and what happens when the queue is full.	Run all producers together and send bursts of data. Record event waiting time, dropped messages and whether control tasks still meet their deadlines.
remote_q	Wireless Comms; Controller	Wireless Comms checks incoming commands; Vehicle Controller decides whether they are allowed in the current state.	Wireless Comms adds valid commands; Vehicle Controller removes and processes them.	Many incoming commands can fill the queue. A command that waits too long may no longer be appropriate when the controller receives it.	Check commands before forwarding them. Set a queue limit and agree when old commands expire, how rejected commands are reported and how receipt is acknowledged.	Send bursts, invalid commands and commands with expired timestamps. Check which commands reach the controller and measure the response to STOP.
motion_q	Controller; Motion Control	Vehicle Controller issues movement requests. Motion Control alone applies them to the motors.	Vehicle Controller adds speed, distance or turn commands; Motion Control reads them.	Old movement commands may remain in the queue after a stop. Executing them during recovery could move the robot unexpectedly.	Motion Control checks the latched STOP flag before acting on any command. The controller discards old movement commands before authorising recovery. Also agree the queue size.	Place several movement commands in the queue, then trigger STOP and RESET. Record motor outputs and confirm that old commands do not restart movement.
scan_q	Controller; Obstacle Scan	Vehicle Controller requests a scan. Obstacle Scan manages the	Vehicle Controller sends scan requests; Obstacle Scan receives	A delayed scan request may be processed after it has been cancelled or after the robot has	Give each request an identifier and an expiry rule.	Send repeated scan requests, cancel one and delay another.

Shared resource	Used by task(s)	Owner task / module	Access type	Contention risk	Mitigation	Evidence needed
		sensor and carries it out.	them and checks whether scanning may start.	changed state. This could produce an irrelevant obstacle profile.	Cancel old requests and pending results when required. Start scanning only after the controller confirms that the wheels have stopped.	Check that only the valid request is processed and no scan starts before confirmed stop.
controller_flags	Obstacle Scan; Motion Control; Controller	Obstacle Scan and Motion Control report events. Vehicle Controller handles them and manages when they are cleared.	Tasks set hazard or fault bits; Vehicle Controller reads them and takes the required action.	Two tasks may report a hazard and a fault close together. Clearing the whole flag group while handling one could erase the other before it is handled.	Assign each event its own bit. Agree which task sets and clears each bit. Clear only the event that has been handled, leaving other pending events visible.	Trigger a hazard and a motor fault together. Record the flags and controller actions to show that both events are recognised and handled.
safety_flags	Controller; Motion Control	Vehicle Controller controls STOP and authorises recovery. Motion Control must obey the flag.	Vehicle Controller sets or clears STOP; Motion Control checks it before executing movement.	Movement could continue if Motion Control ignores STOP. Clearing STOP before recovery checks are complete could also allow an unsafe restart.	Keep STOP set until the controller authorises recovery. Motion Control checks it before executing commands. Only the controller may clear it after the required safety checks.	Trigger STOP while motion commands are arriving. Check motor outputs and wheel movement, then verify that movement remains blocked until the required recovery and restart steps.
robot_state / state_mutex	Controller; Wireless Comms	Vehicle Controller updates robot_state. Wireless Comms reads a protected copy for telemetry.	The controller locks the state while updating it. Wireless Comms uses the same lock while copying it.	Wireless Comms could copy a mixture of old and new state values. Holding the lock during network work could also delay the controller when it needs to update state.	Use the mutex while updating or copying the shared state. Keep this work brief and release the lock before network publishing. Use priority inheritance to limit blocking of higher-priority work.	Update state while telemetry is being published. Check that each copied snapshot is consistent and measure how long the lock is held and whether control deadlines are met.
Encoder counters	Encoder ISRs; Motion Control	Motion Control	Interrupt handlers update wheel counts. Motion	An interrupt may update a counter while	Take a consistent copy of the counters in a short critical section that is	Generate a known number of encoder

Shared resource	Used by task(s)	Owner task / module	Access type	Contention risk	Mitigation	Evidence needed
			Control copies those counts to calculate speed and distance.	Motion Control is reading it. An inconsistent snapshot could give the wrong wheel distance or speed.	safe with interrupts. Process the copy outside it. Interrupt handlers must never wait for a mutex.	pulses at the maximum expected rate. Compare the recorded count with the reference and check that other sensing still works.
Echo timing / echo_done	Echo ISR; Obstacle Scan	Obstacle Scan	The echo interrupt records pulse times and signals echo_done. Obstacle Scan waits for completion and calculates distance.	An echo may arrive late or a previous completion signal may remain pending. The task could then use the wrong pulse to calculate the current distance.	Wait only for a defined timeout. Link the captured pulse to the current trigger and reject late or invalid results. Do not treat a missing echo as proof of a clear path.	Measure known distances, remove the echo and introduce delayed pulses. Log trigger and echo times to check distance calculations, timeout handling and rejection of old results.
GPIO interrupt dispatcher	Encoder ISRs, Echo ISR, proposed IR edge ISRs	Buddy 2 coordinates the shared dispatcher with Buddies 3 (IR) and 5 (ultrasonic).	One GPIO callback dispatches encoder (GP26/27, GP7/28), ultrasonic ECHO (GP3), and proposed barcode IR edges (GP4/16/6).	Excessive or overlapping GPIO interrupts may delay handlers, causing missed encoder counts, barcode edges or inaccurate echo timing.	Use one coordinated GPIO dispatcher. Keep ISRs short; record edges/timestamps and perform decoding in tasks. Validate combined interrupt load before adopting edge capture.	Simulate or generate encoder, ultrasonic and IR transitions concurrently at representative maximum rates. Record missed edges, timing errors and control deadline misses.
Shared 5 V supply	Motors; servo; HC-SR04; board supply path	Buddy 2 owns assumption A11. All hardware owners coordinate wiring, load limits and power tests.	Motors, servo and sensors draw power from the same supply while operating together.	Motors and the servo can draw extra current when starting, moving or stalling. The shared supply voltage may fall, causing a board reset or unreliable sensor readings.	Check whether the supply and wiring can support the combined load. Test motors and servo together, including stall conditions. Change the power arrangement if voltage drops or resets occur.	Monitor supply voltage and resets while applying motor-stall load and sweeping the servo, with WiFi active. Record sensor behaviour and check against the hardware supply limits.

Subsystem Interface and Code Structure Map

Enter at least **one interface or module per buddy**. Each row should identify who provides the module, who consumes it, what inputs and outputs it exposes, and how it will be validated.

Module / file / interface	Provider	Consumer(s)	Purpose	Inputs	Outputs	Interface rule	Validation rule
WiFi command and telemetry interface	Buddy 1	Vehicle Controller; MQTT client	Receive remote commands and publish robot status.	Broker commands; robot_state snapshot; connection status.	Checked commands through remote_q; telemetry and heartbeat messages.	Wireless Comms checks message format; Controller checks whether the command is allowed in the current state. Agree topics, fields, units, expiry and rejection rules. Release state_mutex before publishing.	Send valid, invalid, old and repeated commands. Check forwarding and state-based rejection. Disconnect/reconnect WiFi and measure telemetry delay and control timing.
Motion command and feedback interface	Buddy 2	Vehicle Controller; encoder ISRs	Control wheel speed, distance and turns; report motion results.	Speed/distance/turn requests through motion_q; STOP flag; encoder counts.	Motor PWM; motion completion, wheel movement and fault reports.	Only Motion Control writes motor PWM. Check STOP before acting. Agree command identifiers, units, limits and completion/fault fields; discard stale commands during recovery.	Compare commanded distance and turns with measurements. Trigger STOP with queued motion; confirm no stale restart. Check encoder counts and control timing under load.
Line and barcode interface	Buddy 3	Vehicle Controller; shared GPIO dispatcher	Report line position, barcode results and junction/line-return information.	Three digital IR inputs; calibration settings; edge times or encoder snapshots for proposed barcode capture.	Line position/validity; decoded command; line-loss, junction and line-return events.	No direct motor calls. Define timestamps, valid/invalid readings and event fields. Agree junction recognition and hold a barcode command until its junction. Define confirmation of the original line.	Compare readings with known line positions and barcode patterns at a stated speed. Test lighting changes, junction command timing and line return after a bypass.
IMU and terrain interface	Buddy 4	Vehicle Controller; telemetry through robot_state	Report motion/terrain events and retain the highest hump peak.	GY-511 readings; calibration values; controller-authorised new-run reset.	Terrain/readiness/fault events; hump-height estimate and run maximum.	Only IMU/Terrain accesses I2C0. Define sample time, units, validity and event fields. Document the height	Compare estimates with measured humps and test flat-surface vibration.

						method and acceptable error. Sensor recovery must not erase the run maximum or restart motion.	Check invalid-reading suppression, reset rules and sampling while other tasks run.
Obstacle scan and bypass interface	Buddy 5	Vehicle Controller; Echo ISR; Motion/Line through Controller	Measure clearance, profile obstacles and propose a bypass.	Scan_q requests; echo timestamps and echo_done; mission state; confirmed stop/cancellation.	Clearance and bypass results through sensor_q; urgent hazard/fault flags.	Only Obstacle Scan controls the ultrasonic trigger and servo. Require confirmed stop before scanning. Agree request IDs, timestamps, distance/angle units, validity, plan fields and cancellation. Controller commands motion and confirms original-line return with Line Sensor.	Test known obstacles, missing/late echoes, cancellation and no-safe-route cases. With Buddies 2 and 3, verify whole-robot clearance, safe stop and return to the original line.

Week 6 Assumption Closure

Enter at least **one assumption per buddy**, preferably copied from Week 6. Each row should show whether the assumption is still open, validated, deferred, or converted into a risk.

Week 6 assumption ID	Original assumption	Current status	Evidence collected	Remaining risk	Next action
A2 (Buddy 1)	Telemetry and reconnection processing can run alongside all other tasks without compromising vehicle-control timing or STOP response.	Open	No test evidence collected yet.	WiFi retries or telemetry work could delay motor control, sensing or STOP handling. A low task priority alone does not prove that timing is safe.	Run telemetry and WiFi reconnection while all control and sensing tasks operate. Measure control cycles against 20 ms and stopping/slowing initiation within 100 ms of a confirmed unsafe obstacle. Revise the design if these limits are missed.
A4 (Buddy 2)	Encoder feedback and motion control can execute turns with sufficient repeatability for navigation.	Open	No test evidence collected yet.	Wheel slip, load or unequal motor response could end a turn at the wrong heading or position. Proposed tolerances (±5° for 90° turns, ±10° for U-turns) are not yet agreed.	Agree tolerances with Buddy 3. Repeat Left, Right and U-turns at representative speeds, loads and battery levels. Measure final heading and position, tune PID and retest.
A11 (Buddy 2)	The power supply remains stable during simultaneous motor, servo and WiFi activity.	Open	No test evidence collected yet.	Motor starts or stalls and servo movement may cause voltage drops, resulting in unintended controller resets, unreliable sensor readings or interrupted motion control.	Test motors under load while sweeping the servo with WiFi active. Monitor supply voltage, resets and sensor behaviour. Confirm the shared 5 V supply remains stable or revise the power arrangement if necessary.
A6 (Buddy 3)	The IR sensors can reliably distinguish barcode bars and spaces at the intended operating speed under expected lighting and course conditions.	Open	No test evidence collected yet.	20 ms polling may miss narrow bars at speed and produce an incorrect command. Edge capture also needs checks for noise and interrupt load.	Agree target speed and barcode read-success rate. Test every barcode repeatedly under expected lighting, using timestamped edges or encoder

					snapshots for capture. Compare decoded commands with known patterns and check execution at the correct junction.
A8 (Buddy 4)	Hump-related sensor readings can be distinguished from normal vehicle vibration and movement.	Open	No test evidence collected yet.	Normal vibration may look like a hump, or a real hump may be missed. Sampling rate and detection thresholds have not been validated.	Record readings on flat ground, during turns and over known humps. Compare detections with observations, agree acceptable missed/false detection rates, and tune thresholds and sampling. Validate the height-estimation method separately.
A12 (Buddy 5)	The scan profile supports a bypass with clearance for the whole vehicle and subsequent line reacquisition.	Open	No test evidence collected yet.	A route that looks clear to the ultrasonic sensor may still clip the robot body. The robot may return to another branch rather than the original line.	Agree whole-vehicle clearance and original-line confirmation with Buddies 2 and 3. Test representative obstacle layouts, scan angles and stopping distances. Record bypass clearance and line return; check that no safe route produces a fault instead of movement.

Version B: Live Zoom-Updated Version

Instructions

- During the recorded Zoom session, update this section live or immediately from discussion notes.
- Record what changed from Version A and why. A change without justification is weak evidence.
- Every buddy must contribute at least one challenge, defence, trade-off, or correction.
- Any use of AI during the Zoom session must be screen-shared and visible to all participants at the point it is being discussed.
- If AI-generated content is introduced during the live discussion, the team must explain the prompt used, the output received, and why the output is being considered.
- AI suggestions must be challenged, validated, refined, or rejected through human engineering discussion; "AI said so" is not sufficient justification.

Acceptable evidence examples: GitHub commit link, short test log, video timestamp, oscilloscope or serial monitor screenshot, pin test result, task timing measurement, AI prompt and critique, meeting decision note, or photo of hardware wiring.

AI-use declaration: Do not ask AI to complete this template on behalf of the team. Use AI as a reviewer, challenger, or brainstorming partner. The team confirms that all final entries were reviewed, corrected, and accepted by the team, and that every member can explain the decisions related to their subsystem.

Buddy Contributions During Zoom

Each buddy must complete **one row**. The contribution must be a real technical challenge, defence, trade-off, or correction raised during Zoom, not only an attendance statement.

Buddy	Key challenge, concern, or argument raised	Challenge received	Decision or compromise reached
Buddy 1 (Hong Yih)	For IMU, telemetry including the current terrain status and the highest hump measurement, We need to agree on the units and how invalid measurements are represented.	Khang (Buddy 4) asked if we need to send every sensor reading through MQTT, or just important information like the current terrain status and highest hump height. Sending everything might create unnecessary overhead.	Include current terrain status, like normal, climbing or descending a hump, as well as the highest hump height recorded during the run. Use millimeters (mm) for height and include a validity flag to indicate whether the measurement is reliable. If the sensor fails or the reading is invalid, we should mark it as invalid instead of reporting an incorrect value. We should keep the highest valid hump measurement until a new run starts, even if the sensor temporarily fails. Only publish important information, such as current terrain status, hump detection and highest hump recorded. Use periodic status updates to reduce unnecessary communication and ensure

			that important information is available for monitoring.
Buddy 2 (Dalton)	Argued that STOP should skip remote_q, since a full queue could drop it or leave it stuck behind old commands. Note that Wireless Comms is the lowest priority, so remote STOP is not the main safety path; obstacle and fault flags are.	Hong Yih (Buddy 1) asked what happens if a remote STOP arrives while remote_q is full, and whether STOP needs a separate urgent notification to the Vehicle Controller.	Wireless Comms sets a REMOTE_STOP bit in controller_flags; the controller wakes on it like a hazard, latches STOP and flushes the queues. Setting a bit can't fail and repeated STOPs collapse into one. START and RESET stay in remote_q because they need state checks. Owners: Buddy 1 sets the bit, Buddy 2 handles it.
Buddy 3 (Edmund)	Minimum-bit representation fits most line and barcode data: 3 bits for the IR sensors, 2 for line position, 3 for the barcode command and 3 per state machine. Storing barcode bar durations as 8-bit milliseconds instead of 32-bit timestamps cuts the capture buffer from 128 to 32 bytes. However, this only works if the robot stays above about 50 mm/s over barcodes. Packed data shared between my ISR and the controller could also lose updates if both write the same word.	Hong Yih (Buddy 1) asked For telemetry, what information should your subsystem expose to me? Do we need to transmit every sensor reading, or would it be sufficient to publish the latest line position, detected barcode, and navigation status?	Publish a summary rather than every sensor reading: the latest line position with a validity flag, the last barcode command, the current line-following state (following, lost, junction) and event bits for line lost, barcode error and line reacquired. These fit in about 2 bytes of compact binary payload. Send them with the periodic status update, and immediately on important events such as a barcode decode or line loss. Raw IR readings and bar timings stay local for debugging. The final bit widths and publishing frequency still need to be agreed with Buddy 1.
Buddy 4 (Khang)	Our GY-511 has no gyroscope, so estimating hump height accurately is difficult because acceleration and vibration can affect the readings. I suggested testing accelerometer-based tilt with encoder distance. The new minimum-bit requirement also means we need to reduce how much data we store and send without losing measurement accuracy.	Hong Yih (Buddy 1) asked whether telemetry should include the current terrain status and highest hump height, and how we should handle units and invalid readings.	We proposed reporting the terrain status and highest valid hump height in mm, together with a validity flag and timestamp. Instead of sending every raw IMU reading, we would send important terrain events and summary measurements. Under the new requirement, these should use compact binary fields based on their valid ranges. Invalid readings would be flagged, and the highest valid hump measurement would remain stored until a new run starts. The final bit widths and

			performance still need validation.
Buddy 5 (Timothy)	Minimum-bit representation may reduce memory usage but introduce processing overhead and race conditions when sharing packed ultrasonic data between the ECHO ISR and obstacle scan task.	Hong Yih (Buddy 1) questioned whether obstacle telemetry should include full scanning data or only obstacle status and bypass results.	Prioritise compact binary telemetry containing obstacle status, essential measurements and bypass results. Retain detailed scanning data locally for debugging or on-demand reporting. We should reserve safe ISR-to-task communication and validate memory saving against timing performance.

Live Decision Change Log

Enter at least **3 changes AND 3 confirmed no-change decisions** from the Zoom discussion. Include the change type and evidence link or timestamp whenever possible. One row may state "No change" if the team defended the Week 6 design successfully.

Original idea / Week 6 design	Updated decision	Change type	Reason for change	Evidence link	Owner / follow-up
Publish robot telemetry through MQTT, but the specific IMU data and publishing frequency were not defined.	Publish only essential IMU information, including terrain status, hump detection events and maximum hump height. Raw readings remain processed locally and may be published during debugging.	Clarified	Reduces MQTT traffic, memory usage and processing overhead without affecting local IMU sampling.	Version B: Buddy 4 contribution and Buddy 1 telemetry discussion.	Buddy 1 (Hong Yih) and Buddy 4 (Khang): Agree on telemetry fields, units and publishing frequency
IMU terrain states, hump measurements and telemetry were defined without specific minimum-bit representations.	Use compact binary fields for terrain states, hump measurements, validity flags and IMU events. Decide each field bit width based on its required range and precision.	Clarified	Meets the new requirements while reducing unnecessary memory and message usage. However, smaller fields must not cause overflow, incorrect measurements or unacceptable processing delays.	Change Requirement 1, p. 1; Version B: Buddy 4 contribution.	Buddy 4 (Khang), with Buddy 1 (Hong Yih): Define field ranges, bit widths and binary telemetry format, compare message sizes and verify correctness.
Subsystem 5 reports obstacle and bypass	Publish obstacle status, essential measurements and	Simplified	Buddy 1 challenged the need for full scan	Version B: Buddy 5 contributio	Buddy 5 (Timothy): agree

results; the amount of scan data sent through MQTT was not defined.	bypass results. Keep detailed scans locally for debugging or on-demand reporting.		telemetry. Summary data reduces MQTT traffic while retaining local scan detail.	n and Buddy 1 telemetry challenge.	obstacle telemetry fields with Buddy 1 (Hong Yih).
Subsystem 5 data and telemetry had no agreed minimum-bit format.	Use compact binary fields for obstacle status, essential measurements and bypass results. Final field widths still need validation.	Clarified	Meet the minimum-bit requirement without losing useful measurements. Packing may save memory but adds processing work.	Version B: Buddy 5 minimum-bit concern and compromise.	Buddy 5 (Timothy): define fields and compare memory savings with timing performance.
A dedicated IMU task handles terrain sensing and reports results to the Vehicle Controller. Only the controller decides how the robot should respond.	Keep the existing IMU task structure and controller ownership. Change the data representation rather than redesigning the subsystem.	No change	The new requirement affects how information is stored and shared, not who controls the motors. Keeping responsibilities separate avoids unnecessary changes and reduces integration risk.	Week 6 Design Review: Subsystem 4 Statechart; Version B: Buddy 4 contribution.	Buddy 4 (Khang): Update the IMU data formats and validate event delivery.
Line and barcode results were sent to the controller, but which line and barcode data was published through MQTT, and how often, was not defined.	Publish only a 2-byte summary: line position with a validity bit, last barcode command, line-following state, and event bits (line lost, junction, decode error, line reacquired). Raw IR readings and bar timings stay local for debugging.	Simplified	IR sensors are sampled about 1,000 times a second. Sending every reading would overload low-priority WiFi without helping monitoring.	Version B: Buddy 3 contribution and Buddy 1 telemetry challenge.	Buddy 1 (Hong Yih) and Buddy 3(Edmund) : agree field bit positions, byte order and publishing frequency.
Wireless communication runs at the lowest task priority and does not directly control motor PWM.	Retain the architecture where wireless communication handles MQTT while the vehicle controller retains movement authority.	No change	Preserves separation between non-real-time network communication and time-critical robot control.	Week 6 Design Review: Wireless Comms Task; Version B: Buddy 1 and Buddy 2 discussion.	Buddy 1(Hong Yih) and Buddy 2(Dalton): Validate control timing during Wi-Fi reconnection.
Line and barcode sensing reports results to the	Keep this boundary. Only the data format changes: results are	No change.	The new requirement changes how the	Version B: Buddy 3 contributio	Buddy 3 (Edmund) and Buddy

Vehicle Controller and never writes motor commands directly. The controller holds a decoded barcode command until the junction is reached.	sent as a packed 16-bit message through sensor_q, and shared event bits use T-Kernel event flags.		data is stored, not who controls the motors. Keeping one motor writer avoids conflicting commands.	n and Buddy 1 telemetry challenge.	2 (Dalton): agree the packed message format and junction command timing.

Resource and Assumption Review

Item challenged	Evidence presented	Decision made	Impact on other buddies
Buddy 1: We questioned whether MQTT needs every sensor reading and how much shared-state and network work we can afford.	In our discussion, we identified the IMU, line/barcode and obstacle summaries needed for monitoring. We have no measured traffic or timing results yet.	We will publish essential status, events and measurements, keeping detailed readings local. We will agree field formats and update frequency, then validate timing under WiFi load.	We will agree summary fields across Buddies 1, 3, 4 and 5 and check control and STOP timing with Buddy 2. We will copy the shared state briefly and publish after releasing the mutex.
Buddy 2: We challenged sending remote STOP through remote_q, where pending commands or a full queue could delay or lose it.	In our discussion, we raised the full-queue case and noted that Wireless Comms has the lowest priority. We have not yet measured STOP response under load.	We agreed to signal REMOTE_STOP through controller_flags. Our controller will latch STOP and flush queued commands; START and RESET remain in remote_q for state checks. We retain local hazard and fault signalling as our main safety path.	We need Buddy 1 to set REMOTE_STOP and Buddy 2 to handle it. With Buddy 5, we must check simultaneous hazard/STOP events, queue flushing and recovery without stale movement.
Buddy 3: We challenged the range and concurrency assumptions behind minimum-bit IR and barcode data.	In our discussion, we estimated that 8-bit bar durations could reduce our capture buffer from 128 to 32 bytes, assuming speed stays above about 50 mm/s. We also raised lost updates when writers share a packed word; these are not validated results.	We will use a compact line/barcode status summary and keep raw readings local. We will test duration overflow at the lowest intended speed and check concurrent packed-data updates before fixing the format.	We will agree bit positions, byte order and publishing frequency with Buddy 1, validate speed and shared interrupt load with Buddy 2, and agree line-reacquisition status with Buddy 5.
Buddy 4: We challenged whether our GY-511 can support accurate hump-height estimates and how compact fields affect precision.	In our discussion, we noted that our GY-511 has no gyroscope and that acceleration and vibration affect tilt readings. We suggested combining accelerometer-based tilt with encoder distance; we have not validated the method.	We will report terrain status, highest valid hump height in mm, validity and timestamp fields, keeping raw readings local. We will retain the run maximum until a new run and test the proposed height method against measured humps before choosing field widths.	We will evaluate the height method using Buddy 2's encoder distance and agree compact telemetry fields with Buddy 1. We will check range, invalid readings and timing against our measurement requirements.
Buddy 5: We challenged whether compact ultrasonic shared/queue fields save memory without adding unsafe ISR-to-task races or processing delays.	In our discussion, we raised packing overhead and shared ECHO-data races. With Buddy 1, we questioned full-scan telemetry. We have no measured memory savings or timing validation yet.	We will send obstacle status, essential measurements and bypass results, keeping detailed scans local. We will define ranges and precision before choosing bit widths, preserve safe ISR-to-task transfer, and	We will agree telemetry fields with Buddy 1, test stop/bypass timing and shared interrupts with Buddy 2, and confirm original-line return with Buddy 3. We will test whole-vehicle clearance and line reacquisition to address A12.

		compare packed memory usage and timing under load.	

Risk Assessment and Subsystem Test Plan

Integration Risk Register

Enter at least **3 integration risks**. Include at least one timing or scheduling risk, one hardware or wiring risk, and one interface or team-integration risk.

Risk ID	Risk	Cause	Impact	Likelihood	Severity	Mitigation	Owner	Status
R1	Wi-Fi reconnection affects control timing	Network reconnection consumes CPU time or causes shared-resource blocking.	Motor control may miss its 20 ms cycle, affecting movement and safety response.	Medium	High	Keep Wireless Communication at low priority, minimise mutex holding time and test reconnection under full system load.	Buddy 1 & 2	Open
R2	Barcode bar timings are measured wrongly, or encoder counts are lost.	Barcode, encoder and echo interrupts share one GPIO handler and can delay each other at high speed.	Wrong barcode command or an incorrect turn and distance.	Medium	High	Keep handlers minimal (timestamp only), measure interrupt load at maximum speed, reject failed decodes, and fall back to 1 kHz timer polling if needed.	Buddy 3, with Buddy 2 and Buddy 5	Open
R3	Unstable power supply	Motors and servo draw too much power when running together	Voltage drops may reset the board or affect the sensors.	Medium	High	Test motors, servo and WiFi together. Monitor voltage and unexpected resets.	Buddy 2, 5	Open
R4	Incorrect data shared between subsystems	Buddies use different bit sizes, layouts or units.	The controller or MQTT may receive incorrect information.	Medium	High	Agree on one data format and test that messages are encoded and decoded correctly	All buddies	Open
R5	Inaccurate hump measurements	Too few bits are used to store hump heights.	Height measurements may lose precision or exceed the allowed range.	Medium	Medium	Agree on the required range and precision. Values outside the range are capped and flagged as invalid. Test	Buddy 4	Open

						against known hump heights.		
R6	Race conditions when updating	Multiple tasks update different bits in the same shared fault bitmask without proper synchronisation.	Fault information may be overwritten or lost, affecting fault handling and robot safety.	Medium	High	Use atomic operations or appropriate synchronisation when updating shared packed data. Test simultaneous fault updates to ensure no information is lost.	Buddy 2 with all buddies	Open

Subsystem Test Plan

Enter **at least 4 cross-subsystem tests**. Each test must involve the work of **at least two buddies** and must include a success criterion and evidence type, such as a video, serial log, screenshot, timing result, or commit link. At least **TWO** test should involve timing, safety, or shared-resource concern.

Subsystem	Test case	Success criteria	Evidence required	Owner	Status
IMU + Wifi (B4, B1)	Send packed terrain status, height, validity and timestamp through MQTT.	Correct decoding of valid and invalid messages, including boundary values and message size compared with original format	Binary message capture, decoded log, before/after size comparison	Buddy 4, 1	Open
Line/Barcode + Motion (B3, B2)	Drive over each barcode at the minimum and maximum target speeds while the ISR and controller both update the shared event flags. Check that the packed command is held until the junction.	Correct command decoded on at least 95% of passes. The turn is executed at the correct junction. No event bits are lost or overwritten. The 8-bit durations never saturate mid-barcode.	Serial log of decoded command, event flags and timestamps; video of the turn at the junction; before/after buffer size comparison	Buddy 3	Open
Motion + Obstacle + WiFi (B2, B5, B1)	Run motors at max speed with packed 16-bit encoder counters while the servo sweeps, echo ISR captures pulses, and WiFi reconnects. Send a remote STOP with remote_q full.	Encoder counts match reference within ±1% with no lost counts; control loop stays within 20 ms with packed data; REMOTE_STOP latches and motors are commanded off within one 20 ms cycle; no stale command restarts motion.	Serial log of encoder counts vs reference, control loop timing before/after packing, STOP timestamp log, video of wheels stopping.	Buddy 2	Open

Obstacle Scan + Motion + Line + WiFi (B5, B2, B3, B1)	We will scan known obstacles after confirmed stop, then bypass and reacquire our original line. Inject missing/late echoes and no-safe-route cases under WiFi load; compare packed fields with unpacked data and publish summary telemetry.	We pass when the whole robot clears the obstacle and the original line is confirmed; invalid echoes or no safe route keep it stopped. Packed boundary/invalid values decode correctly, with no lost or torn ECHO ISR/task updates. RAM and payload sizes decrease; control remains within 20 ms. Telemetry preserves essential measurements, status and bypass results; detailed scans stay local.	Our bypass/line-return video; timestamped echo and fault logs; packed-field boundary tests; before/after RAM, payload-size and timing measurements; captured MQTT messages and decoded output.	Buddy 5, with B2, B3, B1	Open
Wi-Fi/MQTT + Motor Control	Disconnect and reconnect Wi-Fi while the robot is performing line-following and motor-control operations.	Motor control continues meeting its 20 ms task period, and reconnection does not unexpectedly restart or interrupt movement.	Serial logs showing control task timing, Wi-Fi status and motor behaviour.	Buddy 1, 2	Open

Constraint Injection / Curveball Response

Instructions

- Record the injected constraint exactly as given.
- Explain how it affects all five buddies, even if the impact differs by subsystem.
- A good response shows prioritisation, trade-off reasoning, fallback behaviour, and evidence-aware decision making.

Injected constraint	Buddy affected	Immediate impact	Adaptation / compromise
The robot car firmware must minimise the number of bits used to represent application data. Each data field representing application information must use only the number of bits required to represent its valid range, using C bit-fields or explicit masks and shifts where appropriate.	Buddy 1	We must compact MQTT commands, telemetry and shared robot state without delaying control.	We will publish binary summaries with agreed ranges, units, bit positions and byte order. Raw readings stay local. We will copy state under the mutex, then publish outside it. Invalid commands are rejected; reconnecting cannot restart motion. We will compare payload sizes and control timing under WiFi load.
Motion, encoder and controller data; shared fault bitmask.	Buddy 2	We must reduce counter, state and queue fields while preserving range, STOP priority and safe concurrent updates.	We retain one motor writer and urgent REMOTE_STOP through controller_flags, outside remote_q. We will use individual fault bits with atomic or synchronised updates and

			ISR-safe counter snapshots. We will test overflow, simultaneous faults and full-queue STOP against the 20 ms control target; unsafe cases keep STOP latched.
Line/barcode data, ISR/task sharing and queue messages.	Buddy 3	Our compact durations and status fields may overflow or lose updates when writers share a packed word.	We will send the agreed compact line/barcode summary and keep raw captures local. We will validate the proposed 8-bit durations at the lowest speed, increasing width if the required range needs it. We will test event flags for lost updates, barcode correctness and command execution at the correct junction.
Hump estimation, terrain state and telemetry.	Buddy 4	We must retain useful height precision, validity and timestamps while reducing storage and MQTT size.	We will report terrain status and highest valid hump height in mm, with validity and timestamp fields. We will choose widths from tested ranges and precision, retain the maximum until a new run, and flag invalid readings. We will compare packed decoding and the proposed tilt/encoder method against measured humps.
Obstacle data, queues and shared ECHO ISR/task fields.	Buddy 5	Our compact measurements and bypass results save space but add packing overhead and race risks.	We will send essential obstacle/bypass summaries and keep full scans local. We will use safe ISR/task transfer and test range boundaries, timing, clearance and original-line return. Missing/invalid echoes or no safe route keep us stopped. For Week 13, we will put our field/range/bit-width/implementation table, measured before/after RAM and message sizes, and correctness/timing evidence in GitHub.

Integration Readiness Self-Assessment

Complete all rows. Use **Green** only if there is evidence, **Yellow** if partly ready or untested, and **Red** if the area is unclear or blocking Week 10 integration.

Readiness area	Status	Evidence	Concern
Hardware wiring known	Green / Amber / Red	All components are connected. Hardware photos, schematic diagrams and individual component tests support the connections.	Shared power stability and simultaneous operation still need testing.
RTOS task ownership clear	Green / Amber / Red	Week 6 task table, interaction diagram and statechart clearly define task responsibilities.	Task timing and ownership still need to be verified during implementation.

		Only Motion Control writes motor PWM.	
Shared resources identified	Green / Amber / Red	Shared queues, GPIO interrupts, mutexes and safety flags are documented.	Packed data may introduce race conditions. Synchronisation needs validation.
Critical statecharts implementable	Green / Amber / Red	Week 6 statecharts cover normal operation, STOP, faults and recovery.	Some algorithms and minimum-bit data formats remain untested.
Safety or fault handling defined	Green / Amber / Red	Latched STOP, fault reporting and controller-authorised recovery are documented.	Shared fault bitmask updates and STOP response need testing.
Week 10 partial integration feasible	Green / Amber / Red	Hardware connections, task ownership and subsystem interfaces are documented.	Binary formats, shared-data safety and integration timing require validation.

Updated readiness position before the Week 10 partial integration demo:

Our team has defined the main subsystem responsibilities, interfaces and safety design. The new minimum-bit requirement means we must also agree on field sizes, binary message formats and safe access to shared packed data.

Before Week 10 integration, we need to finalize the data representations and test that they do not affect measurement accuracy, task timing or fault handling. We will also compare message sizes and RAM usage before and after the changes.

Overall, we believe that partial integration is indeed feasible, but these risks will still need to be validated. Therefore, we aim to keep our existing architecture where possible to avoid unnecessary changes and integration issues.

Version C: Final AI-Vetted Gap Check

Instructions

- After the live discussion, give the updated Version B content to AI and ask AI to act as a strict embedded systems reviewer.
- Ask AI to identify missing assumptions, weak evidence, unclear interfaces, untested failure modes, and resource conflicts.
- For every AI-identified gap, decide whether to fix, reject, or defer it, and explain why.

AI reviewer prompt used	Act as a strict embedded-systems reviewer of our updated Version B. Identify missing assumptions, weak evidence, unclear interfaces, untested failure modes and resource conflicts under the minimum-bit requirement. Distinguish documented decisions from measured results. Propose justified fix, reject or defer responses without inventing evidence.

AI-identified gap	Team response	Action taken	Evidence / reason
STOP timing and decision traceability are unclear.	Fix. Bypassing remote_q avoids queue delays, but network and task delays are still possible. We will measure the time from receiving STOP to setting the flag, and from setting the flag to turning off the motors.	We recorded the missing STOP decision in Version C. Remote STOP bypasses remote_q through controller_flags. The controller then latches STOP and clears old commands. Buddies 1 and 2 will test the response time, including when remote_q is full.	Buddies 1/2 agreed to the bypass in B, but it is missing from the change log. A's 20 ms target starts when STOP is set; network delay is unproven.
Packed field ranges and interfaces remain incomplete.	Fix. Agree on valid ranges, precision and bit layouts before packing data to prevent overflow and ensure all subsystems interpret the values correctly.	We identified the remaining field requirements. Buddy 2 will check encoder limits, snapshot timing and overflow. We will test boundary values and message decoding before finalising the formats.	B leaves field widths unresolved. At 1:10:25, we required range and resolution before selecting bits. Counter width alone does not prove safe operation.
Packed-data and event updates could lose information.	Fix. Define who can update shared data and ensure safe ISR-to-task communication. Test simultaneous events to prevent lost information.	We will capture ECHO timestamps in the ISR, pack in the scan task, and test simultaneous events, full queues and event clearing.	B identifies shared-word races. At 1:11:36 and 1:13:16-1:14:55, we discussed ISR/task separation and urgent hazards bypassing ordinary queues.
Eight-bit barcode durations and the proposed 1 kHz sampling rate have not been validated.	Fix. We need to confirm the actual sampling rate and check whether 8-bit durations can represent the longest barcode bars at the lowest operating speed.	We will measure the sampling rate, CPU load and barcode timing under RTOS load. If 8 bits are insufficient, we will increase the field width. Buddies 2 and 3 will also confirm when barcode commands should be executed.	Week 6 did not clearly define when barcode commands should be executed, although Version B listed this as unchanged. The proposed 1 kHz sampling rate and 8-bit durations, assuming a minimum speed of about 50 mm/s, also need testing.
Compact hump telemetry does not establish height accuracy.	Defer accuracy claims until we test the tilt-and-encoder method against known humps.	Buddy 4 will test the tilt-and-encoder method using humps of known heights, check false detections	B acknowledges the GY-511 has no gyroscope and the estimation method is unvalidated. Smaller

	Check vibration, false detections and sampling before finalising bit widths.	caused by vibration and verify that packed values keep enough precision. We will also check IMU sampling while other RTOS tasks are running.	fields cannot correct measurement errors.
Obstacle clearance and original-line recovery lack measurable criteria.	Fix. We will agree whole-vehicle clearance and original-line confirmation with Buddies 2 and 3.	We will define scan and settling timings, then test missing echoes, blocked routes and ambiguous branches while retaining stop-before-scan.	B states safe outcomes but gives no numerical clearance or line-confirmation rule. Our discussion retained controller movement authority.
Combined-load safety has not been verified through testing.	Defer readiness claims until integrated tests confirm control timing, event delivery and power stability while all subsystems run together.	We will combine encoder, barcode, ECHO, servo and WiFi activity; measure control/STOP timing, event loss and supply stability.	Version B includes combined-load tests, but they remain Open. We still need actual timing, event-loss and power measurements to confirm safety.
Memory savings and readiness are not yet evidenced.	Defer memory and performance improvement claims until testing. We reject the assumption that using fewer bits automatically improves performance.	We will compare RAM usage, queue and message sizes, and task timing before and after packing the data. We will only claim improvements when the measurements support them.	At 1:14:55–1:15:50, we agreed that memory savings and performance improvements must be measured before making claims. Version B uses colour highlighting for readiness, but RAM and timing improvements remain unverified.

5. Final Tribunal Summary

Final question	Team answer
What is the strongest part of the current design?	We retain clear control ownership: the Vehicle Controller authorises movement and recovery, and only Motion Control writes motor PWM. Latched STOP and stop-before-scan provide a consistent safety structure, though timing still needs validation.
What is most likely to fail?	Our highest risk is integration under simultaneous interrupt, WiFi and actuator load. Packed-data races, counter overflow or lost events could delay control or cause incorrect decisions. Power drops could also reset the robot.
What evidence supports readiness?	Our Week 6 task tables, statecharts and interface plans support architectural readiness. Version B records hardware connections and component checks, but integrated tests remain Open and linked timing, power and memory measurements are still needed. We cannot yet claim validated integration readiness.
What did AI miss that human discussion revealed?	AI gave general advice about data packing and safety, but our discussion clarified the details. We agreed to choose bit widths based on valid ranges, capture ECHO timestamps in the ISR before packing data in the task, and send urgent hazards outside normal queues. We also agreed to send compact telemetry summaries instead of every raw sensor reading.
What did AI correctly identify after discussion?	AI correctly highlighted the missing STOP change-log entry and unclear timing origin, incomplete packed-field contracts and encoder overflow/wraparound handling, unvalidated barcode and hump assumptions, and missing combined-load tests and measured savings. We will close these gaps before claiming readiness.